

Visió per Computador

Classificació d'imatges i identificació de cucs

NEUS OLLER MATAS

*Enginyeria Informàtica
Universitat Rovira i Virgili*

Index

1	Introducció.....	3
2	Imatges a tractar.....	3
3	Disseny de la solució	4
3.1	Pseudocodi	4
3.2	Decisions de disseny.....	5
3.2.1	Preprocessament	5
3.2.2	Classificació dels cucs.....	7
3.3	Presentació dels resultats	8
4	Joc de proves	8
5	Conclusions	10
6	Bibliografia	10

1 Introducció

En aquesta pràctica, es desenvoluparà un programa capaç de classificar imatges obtingudes amb un microscopi i identificar cucs intestinals presents en aquestes imatges. La classificació es basarà en dos categories principals: **cucs vius** i **cucs morts**.

Els cucs vius es caracteritzen per una forma curvilínia, mentre que els cucs morts presenten una forma rectilínia. El programa no només haurà de determinar la presència de cucs vius o morts en cada imatge, sinó també quantificar el nombre de cada tipus.

La classificació de les imatges es farà en funció de la majoria dels cucs presents: si la majoria són vius, la imatge es classificarà com a "vius"; si la majoria són morts, es classificarà com a "morts".

2 Imatges a tractar

Abans de començar amb el disseny implementat i la seva evolució, vaig dur a terme un anàlisi de les imatges proporcionades. D'aquesta manera he pogut entendre quines tècniques i mètodes podria aplicar millor i així, fer que sigui més efectiu per a la classificació i identificació dels cucs vius i morts.

L'anàlisi es va fer a totes les imatges per assegurar que es tenien en compte totes les característiques possibles. Però, en aquesta documentació, explicaré l'anàlisi utilitzant dues imatges de referència, ja que considero que són representatives i ajuden a entendre les característiques generals observades. Aquestes imatges són "wormA03.tif" i "wormA24.tif".

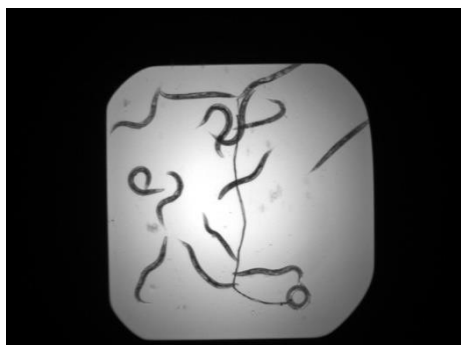


Figura 1: wormA03.tif



Figura 2: wormA24.tif

Podem extreure que totes les imatges a tractar tenen la mateixa estructura; un fons relativament uniforme, fet que facilita la detecció dels cucs. El contrast entre els cucs i el fons permet diferenciar-los clarament, afavorint una detecció més precisa.

En la Figura 1 es poden veure diverses formes curvilínies com a exemples que poden tenir els cucs vius i a la Figura 2 es veuen les formes rectilínies que poden tenir els cucs morts.

3 Disseny de la solució

3.1 Pseudocodi

Després d'analitzar les imatges i comprendre les seves característiques, aquest és l'esquema de la solució proposada.

```
imatges := LlegirImatges(carpeta_imatges);

area_minima = 50
umbral_circularity = 0.18

Per cada i de 1 a total_imatges fer

    imatge := LlegirImatge(imatges[i])

    imatge_normalitzada := adapthisteq(imatge);

    threshold := graythresh(imatge_normalitzada);

    imatge_binaria := imbinarize(imatge_normalitzada, threshold);

    imatge_binaria_processada := bwareaopen (imatge_binaria);

    imatge_binaria_processada := bwareaopen(imatge_binaria, 100);

    contorns, Etiqueta := bwboundaries(imatge_binaria_processada);

    propietats := regionprops(Etiqueta, 'Area', 'Perimeter', 'Centroid');

    Per k := 1 a contorns Fer

        contorn := contorns{k};
        area_contorn := propietats(k).Area;
        perimetre_contorn := propietats(k).Perimeter;

        Si area_contorn >= area_minima) i area_contorn < 5000 Llavors

            circularity := (4 * pi * area_contorn) / (perimetre_contorn ^ 2);
            centroid := propietats(k).Centroid;

            Si circularity < umbral_circularity Llavors

                morts := morts + 1;
                categoria := 'Mort';
                color_categoria = 'red';

            Sinó

                vius := vius + 1;
                categoria := 'Viu';
                color_categoria := 'green';

        FSi

    FSi

Final Bucle
```

3.2 Decisions de disseny

3.2.1 PREPROCESSAMENT

Aquest programa ha requerit tenir en compte diverses decisions en quan fa a la seva implementació.

Per acabar de poder identificar tots els cucs en el seu contrast de la imatge, aplicarem una normalització per tal de poder acabar de definir les petites regions dels cucs que no s'acaben de diferenciar amb el fons.

Dintre de les diferents normalitzacions:

- Normalització de contrast global: `imadjust`
- Equalització d'histogrames: `histeq`
- Normalització d'histogrames adaptativa: `adapthisteq`
- Normalització per llindar o per percentil: `stretchlim`
- Normalització de color: `imhistmatch`

Utilitzo la funció “*adapthisteq*” la qual ens acabarà d'ajustar el contrast d'aquestes regions específiques i a més és una normalització útil quan es tracta d'àrees amb diferents nivells de brillantor a la imatge, cosa que és el nostre cas amb tots els fons de les diferents imatges, que tenim diferents nivells de grisos.

```
imatge_normalitzada = adapthisteq(imatge);
```



Figura 3: Imatge normalitzada d'histogrames adaptativa

El següent pas pel preprocessament, és la binarització de la imatge. Per poder-ho dur a terme, abans però, s'ha de trobar el valor del llindar. La millor manera és amb el mètode OSTU, aquest determina un valor òptim de llindar binari per convertir la imatge normalitzada en una imatge binària.

La funció “*graythresh*” aplica el mètode OSTU determina un valor de llindar binari per poder convertir la imatge normalitzada en una imatge binària. Aquest llindar minimitza la variància intra-classe dels píxels de la imatge, i al mateix temps, maximitza la variància entre les classes de píxels.

Una vegada trobat el llindar, aplicarem la imatge normalitzada anteriorment amb aquest llindar per convertir la imatge en binària, on els píxels que tenen una intensitat superior al llindar s'assignen a blanc, mentre que els que tenen una intensitat inferior s'assignen a negre.

```
threshold = graythresh(imatge_normalitzada);  
imatge_binaria = imbinarize(imatge_normalitzada, threshold);
```

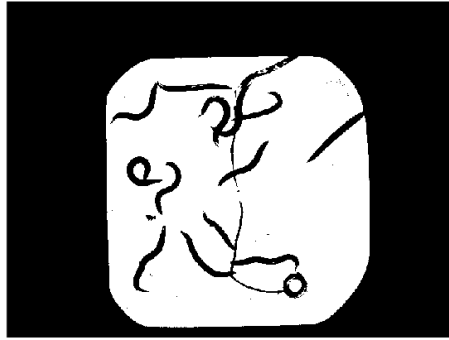


Figura 4: Imatge binaritzada

Una vegada binaritzada la imatge, utilitzo la funció “*bwareaopen*” per eliminar els objectes que tenen una àrea inferior a 100. Això ho faig perquè al binaritzar la imatge es poden potenciar petits objectes que no són rellevants.

```
imatge_binaria_processada = bwareaopen(imatge_binaria, 100);
```

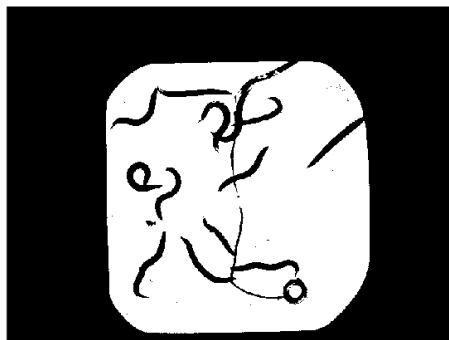


Figura 5: resultat d'eliminar objectes poc rellevants

Ara ja es poden detectar els objectes i per fer-ho, utilitzo la funció “*bwboundaries*” la qual troba els contorns d'objectes en una imatge binària. Tindrem una matriu de cel·les que conté els punts que defineixen els contorns de tots els objectes presents a la imatge binària que s'està tractant en el moment.

Amb aquesta funció podrem extreure dos outputs importants a l'hora de l'anàlisi dels cucs vius i/o morts.

- La primera sortida serà la variable “*contorns*” la qual és un conjunt de matrius. Cada matriu representa els punts que defineixen el contorn d'un objecte trobat a la imatge binària. Cada fila de la matriu representa un punt del contorn, i la primera i última columna contenen les coordenades (*x*, *y*) d'aquests punts.
- La segona sortida serà la variable “*Etiqueta*” la qual és una matriu. Aquesta matriu té la mida de la imatge binària processada (“*imatge_binaria_processada*”), però cada píxel té assignada una etiqueta que indica a quin objecte pertany. L'etiqueta és un número enter positiu que identifica l'objecte al qual pertany el píxel, i els píxels que no formen part de cap objecte tenen un valor de zero.

```
[contorns, Etiqueta] = bwboundaries(imatge_binaria_processada);
```

En la fase final del preprocessament, utilitzaré la funció “*regionprops*” ja que calcula les propietats indicades de les regions etiquetades en una imatge binària.

Aquestes propietats calculades s'utilitzen més endavant al codi per a la classificació dels cucs com a vius o morts.

```
propietats = regionprops(Etiqueta, 'Area', 'Perimeter', 'Centroid');
```

3.2.2 CLASSIFICACIÓ DELS CUCS

Per a la classificació dels cucs en vius o morts, he utilitzat la mètrica de circularitat de cada cuc. La circularitat és una mesura que ens ajuda a determinar la forma d'un objecte. Aquesta mètrica es calcula a partir de l'àrea i el perímetre de l'objecte, propietats que he extret prèviament.

- **Àrea:** L'àrea d'una regió és la quantitat de píxels que ocupa. És una mesura de la mida de l'objecte.
- **Perímetre:** El perímetre d'una regió és la longitud de la seva frontera. És una mesura de la longitud de la línia que delimita l'objecte.
- **Centroide:** El centroide d'una regió és el seu punt mitjà i representa la seva posició en coordenades (x, y).

Càlcul de la circularitat

La circularitat es calcula amb la fórmula següent:

$$Circularity = \frac{4\pi * Area}{Perimetre^2}$$

Una circularitat de 1 indica que l'objecte és un cercle perfecte. En el context dels cucs, com més alta sigui la circularitat, més curvilini serà el cuc. Per tant, un cuc viu, que té una forma més sinuosa, tindrà una circularitat més alta, mentre que un cuc mort, que tendeix a ser més rectilini, tindrà una circularitat més baixa.

Si fem referència al codi, analitzaré el següent tros:

```
if ((area_contorn >= area_minima) && area_contorn < 5000)

    circularity = (4 * pi * area_contorn) / (perimetre_contorn ^ 2);
    centroid = propietats(k).Centroid;

    if circularity < umbral_circularity
        morts = morts + 1;
        categoria = 'Mort';
        color_categoria = 'red';
    else
        vius = vius + 1;
        categoria = 'Viu';
        color_categoria = 'green';
    end
end
```

Primer de tot, filtro els objectes basant-se en la seva àrea. Només es consideren els objectes amb una àrea més gran que un valor mínim (*area_minima*) i menor que un llindar superior (5000). Això ajuda a eliminar objectes que són massa petits per ser cucs i possibles artefactes grans que no són rellevants.

Després de filtrar els objectes, es calcula el valor de la circularitat de l'objecte utilitzant la fórmula esmentada anteriorment. A més, es determina el centroid de l'objecte, que és el punt mitjà de l'objecte en coordenades (x, y). Aquesta informació s'utilitza més endavant per etiquetar visualment els objectes.

3.3 Presentació dels resultats

Una vegada classificat el cuc tractat, es presentarà de la següent manera:

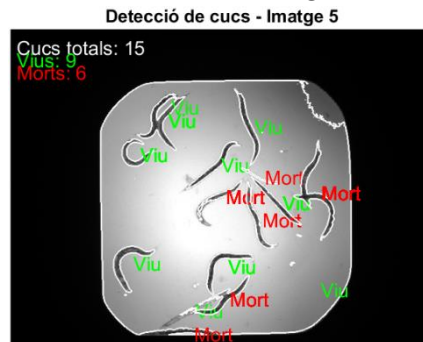


Figura 6: Output de la imatge

```
plot(contorn(:,2), contorn(:,1), 'Color', 'white', 'LineWidth', 1);
text(centroid(1) + 10, centroid(2), categoria, 'Color', color_categoria, 'FontSize', 12);
```

Aquestes dues línies de codi les utilitzo per a la visualització dels resultats de les imatges.

Concretament, aquestes línies dibuixen els contorns dels objectes detectats a la imatge i afegeixen text que indica la classificació de cada objecte (“Viu” o “Mort”) al centroid calculat.

La primera línia l'utilitzo per dibuixar el contorn de cada objecte detectat a la imatge binària. La variable “*contorn*” conté les coordenades (x, y) dels punts que formen el contorn d'un objecte. La funció *plot* agafa aquestes coordenades (“*contorn(:,2)*” i “*contorn(:,1)*”) i les dibuixa sobre la imatge original.

La segona línia l'utilitzo per afegir text a la imatge, concretament indico si el cuc està viu o mort.

4 Joc de proves

El joc de proves ens servirà per poder avaluar la precisió del programa. Les proves consistiran en utilitzar el fitxer *WormDataA.csv* proporcionat. Aquest fitxer conté els resultats esperats per cada una de les imatges.

El que he realitzat ha sigut treure per pantalla els resultats obtinguts amb els meu anàlisi juntament fent una comparativa amb aquests resultats del fitxer. A més a més, en cada execució guardo les imatges en una carpeta anomenada “*resultats*” i, també he creat una fulla de càlcul amb tots els resultats de les imatges guardades.

A continuació es pot veure la sortida per terminal amb tots els resultats. Comentar que encara que els resultats individuals de cucs no són molt precisos, considero que la classificació final és bastant bona i fiable si s'enfoca en un punt de vista general i no individual per cada cuc.

Imatge 1:

1. Cucs totals -> 12 esperats -> 14
2. Cucs Vius -> 9 esperats -> 12
3. Cucs Morts -> 3 esperats -> 2

Imatge 2:

1. Cucs totals -> 10 esperats -> 15
2. Cucs Vius -> 7 esperats -> 12
3. Cucs Morts -> 3 esperats -> 3

Imatge 3:

1. Cucs totals -> 8 esperats -> 15
2. Cucs Vius -> 6 esperats -> 14
3. Cucs Morts -> 2 esperats -> 1

Imatge 4:

1. Cucs totals -> 8 esperats -> 15
2. Cucs Vius -> 4 esperats -> 13
3. Cucs Morts -> 4 esperats -> 2

Imatge 5:

1. Cucs totals -> 15 esperats -> 15
2. Cucs Vius -> 9 esperats -> 13
3. Cucs Morts -> 6 esperats -> 2

Imatge 6:

1. Cucs totals -> 11 esperats -> 15
2. Cucs Vius -> 8 esperats -> 11
3. Cucs Morts -> 3 esperats -> 4

Imatge 7:

1. Cucs totals -> 8 esperats -> 14
2. Cucs Vius -> 4 esperats -> 12
3. Cucs Morts -> 4 esperats -> 2

Imatge 8:

1. Cucs totals -> 12 esperats -> 14
2. Cucs Vius -> 5 esperats -> 14
3. Cucs Morts -> 7 esperats -> 0

Imatge 9:

1. Cucs totals -> 16 esperats -> 16
2. Cucs Vius -> 11 esperats -> 13
3. Cucs Morts -> 5 esperats -> 3

Imatge 10:

1. Cucs totals -> 14 esperats -> 17
2. Cucs Vius -> 10 esperats -> 15
3. Cucs Morts -> 4 esperats -> 2

Imatge 11:

1. Cucs totals -> 13 esperats -> 15
2. Cucs Vius -> 9 esperats -> 14
3. Cucs Morts -> 4 esperats -> 1

Imatge 12:

1. Cucs totals -> 13 esperats -> 15
2. Cucs Vius -> 6 esperats -> 15
3. Cucs Morts -> 7 esperats -> 0

Imatge 13:

1. Cucs totals -> 12 esperats -> 14
2. Cucs Vius -> 4 esperats -> 5
3. Cucs Morts -> 8 esperats -> 9

Imatge 14:

1. Cucs totals -> 13 esperats -> 14
2. Cucs Vius -> 3 esperats -> 2
3. Cucs Morts -> 10 esperats -> 12

Imatge 15:

1. Cucs totals -> 12 esperats -> 15
2. Cucs Vius -> 1 esperats -> 2
3. Cucs Morts -> 11 esperats -> 13

```

-----
Imatge 16:
1. Cucs totals -> 6 esperats -> 13
2. Cucs Vius -> 0 esperats -> 2
3. Cucs Morts -> 6 esperats -> 11
-----
Imatge 17:
1. Cucs totals -> 9 esperats -> 14
2. Cucs Vius -> 2 esperats -> 1
3. Cucs Morts -> 7 esperats -> 13
-----
Imatge 18:
1. Cucs totals -> 11 esperats -> 14
2. Cucs Vius -> 2 esperats -> 1
3. Cucs Morts -> 9 esperats -> 13
-----
Imatge 19:
1. Cucs totals -> 9 esperats -> 12
2. Cucs Vius -> 1 esperats -> 0
3. Cucs Morts -> 8 esperats -> 12
-----
Imatge 20:
1. Cucs totals -> 13 esperats -> 12
2. Cucs Vius -> 2 esperats -> 2
3. Cucs Morts -> 11 esperats -> 10
-----
Imatge 21:
1. Cucs totals -> 9 esperats -> 17
2. Cucs Vius -> 2 esperats -> 1
3. Cucs Morts -> 7 esperats -> 16
-----
Imatge 22:
1. Cucs totals -> 10 esperats -> 14
2. Cucs Vius -> 2 esperats -> 0
3. Cucs Morts -> 8 esperats -> 14
-----
Imatge 23:
1. Cucs totals -> 13 esperats -> 12
2. Cucs Vius -> 5 esperats -> 0
3. Cucs Morts -> 8 esperats -> 12
-----
Imatge 24:
1. Cucs totals -> 13 esperats -> 14
2. Cucs Vius -> 5 esperats -> 0
3. Cucs Morts -> 8 esperats -> 14
-----
Classificació de les imatges -> 20
Classificació esperada -> 24

```

5 Conclusions

Amb aquesta pràctica he pogut aprendre i investigar per obtenir un coneixement més profund en el processament d'imatges biomèdiques i ha demostrat com les tècniques de processament d'imatges poden ser aplicades de manera efectiva per a la classificació automàtica d'organismes microscòpics.

6 Bibliografia

- I. [Medir propiedades de regiones de imágenes - MATLAB regionprops - MathWorks España](#)
- II. [Conectividad de píxeles - MATLAB & Simulink - MathWorks España](#)
- III. [Buscar y contar los componentes conectados de una imagen binaria - MATLAB bwconncomp - MathWorks España](#)
- IV. [Newest 'computer-vision' Questions - Stack Overflow](#)
- V. [Explore GitHub · GitHub](#)